

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
ERROR HANDLING



Disusun Oleh:
ELISABETH NAINGGOLAN
105224037

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Praktikum Pemrograman Berorientasi Objek (PBO) pada modul ini berfokus pada pemahaman dan implementasi Error Handling di dalam pemrograman Java sebagai mekanisme pertahanan sistem terhadap error tak terduga (runtime errors). Berbeda dengan pendekatan pengkondisian konvensional (if-else) yang menggabungkan logika bisnis dengan kontrol kesalahan, mekanisme Error Handling memisahkan kedua aspek tersebut menggunakan blok try-catch-finally.

Implementasi studi kasus pada praktikum ini adalah perancangan program simulasi Sistem Reservasi Tiket Kereta Api "java express" berbasis pemrograman berorientasi objek yang mengintegrasikan penanganan error berlapis guna menjamin stabilitas aplikasi saat berjalan.

Sistem "java express" dirancang menggunakan kombinasi Custom Exception bertipe Checked dan Unchecked Exception guna memvalidasi data bisnis secara spesifik. Dua komponen Checked Custom Exception, yaitu kegagalan pencarian rute kereta (`RuteTidakDitemukanException`) dan keterbatasan ketersediaan kursi (`TiketHabisException`), diimplementasikan dengan mewarisi kelas Exception dasar. Sementara itu, validasi format Nomor Induk Kependudukan (NIK) penumpang menerapkan Unchecked Exception berbasis `RuntimeException` melalui kelas `DataPenumpangTidakValidException`.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1.	kodekereta	String	menyimpan identitas atau kode unik dari setiap kereta.
2.	namakereta	String	menyimpan informasi nama dari kereta.
3.	rute	String	menyimpan informasi stasiun asal dan stasiun tujuan kereta.
4.	kapasitas	int	menyimpan jumlah total kapasitas kursi kereta saat inisialisasi.
5.	sisakursi	int	menyimpan jumlah kursi yang masih tersedia untuk dipesan.
6.	daftarkereta	List<KeretaApi>	koleksi dinamis berbasis arraylist untuk menampung seluruh data objek kereta.
7.	nik	String	variabel untuk menampung 16 digit nomor identitas penumpang.
8.	namapenumpang	String	parameter input untuk menyimpan nama lengkap dari pemesan tiket.
9.	jumlahtiket	int	parameter input penampung kuantitas tiket yang ingin dipesan.

10.	sistem	pusatKontroReservasi	objek utama sistem reservasi yang diinstansiasi di dalam kelas javaexpress (main).
11.	scanner	Scanner	objek resource stream untuk membaca input data dari pengguna via konsol.
12.	pilihan	int	menyimpan angka menu operasi (1-3) yang dipilih oleh pengguna.
13.	running	boolean	flag kontrol perulangan utama program; bernilai false saat pengguna memilih keluar.
14.	keretadipilih	KeretaApi	objek penampung sementara saat proses pencarian kecocokan kode kereta.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1.	KeretaApi()	Constructor	Menginisialisasi objek model kereta baru yang mencakup kode, nama, rute, kapasitas, dan sisa kursi awal.
2.	getKodeKereta()	Functional	Mengambil nilai atribut privat kodeKereta (Getter).
3.	getNamaKereta()	Functional	Mengambil nilai atribut privat namaKereta (Getter).
4.	getRute()	Functional	Mengambil nilai atribut privat rute (Getter).
5.	getKapasitas()	Functional	Mengambil nilai atribut privat kapasitas (Getter).
6.	getSisaKursi()	Functional	Mengambil nilai atribut privat sisaKursi (Getter).
7.	kurangiKursi()	Procedural	Mengurangi nilai sisaKursi saat ada pemesanan tiket yang berhasil.
8.	toString()	Functional	Mengembalikan representasi String terformat dari objek kereta untuk ditampilkan di jadwal.
9.	DataPenumpangTidakValidException()	Constructor	Menginisialisasi objek unchecked custom exception dengan pesan error terkait ketidaksesuaian data NIK.
10.	RuteTidakDitemukanException()	Constructor	Menginisialisasi objek checked custom exception saat kode kereta tidak ditemukan

11.	TiketHabisException()	Constructor	Menginisialisasi objek checked custom exception dengan membawa nama kereta dan sisa kursi.
12.	getNamaKereta() [TiketHabisException]	Functional	Getter untuk mengambil nama kereta dari objek TiketHabisException.
13.	getSisaKursi() [TiketHabisException]	Functional	Getter untuk mengambil sisa kursi dari objek TiketHabisException.
14.	tambahKereta()	Procedural	Menambahkan objek KeretaApi ke dalam daftar koleksi ArrayList.
15.	getDaftarKereta()	Functional	Mengambil seluruh koleksi objek KeretaApi yang terdaftar.
16.	prosesPemesanan()	Procedural	Memproses logika bisnis reservasi: validasi NIK, pencarian kereta, pemeriksaan kursi, dan pelaksanaan pemesanan.
17.	main()	Procedural	Titik masuk program utama; menjalankan antarmuka CLI dan mengendalikan alur try-catch.
18.	tampilkanMenu()	Procedural	Menampilkan pilihan menu utama ke konsol.
19.	lihatJadwal()	Procedural	Menampilkan seluruh daftar kereta yang tersedia beserta informasi kursi.
20.	pesanTiket()	Procedural	Menangani input pengguna untuk proses pemesanan tiket dan menangkap exception yang muncul.
21.	tutupSistem()	Procedural	Dieksekusi dalam blok finally saat program keluar untuk simulasi penyimpanan sesi.

IV. Dokumentasi dan Pembahasan Code

A. Class DataPenumpangTidakValidException

```
THT > src > J DataPenumpangTidakValidException.java
1 public class DataPenumpangTidakValidException extends RuntimeException {
2     public DataPenumpangTidakValidException(String message) {
3         super(message);
4     }
5 }
```

Penjelasan: Kelas DataPenumpangTidakValidException dirancang sebagai Custom Exception untuk mendeteksi kesalahan format pada data masukan pengguna, khususnya NIK, sebelum diproses lebih jauh oleh sistem.

1. Pewarisan RuntimeException (Baris 1): Penggunaan kata kunci extends RuntimeException mengategorikan kelas ini sebagai Unchecked Exception. Artinya, compiler Java tidak mewajibkan penanganan eksplisit melalui blok try-catch atau deklarasi throws pada pemanggil metode, sehingga memberikan fleksibilitas tinggi dalam arsitektur kode.
2. Mekanisme Konstruktor dan Pengiriman Pesan (Baris 2-4): Konstruktor menerima parameter String message yang berisi deskripsi penyebab error. Pemanggilan super(message) meneruskan pesan tersebut ke konstruktor RuntimeException sehingga pesan dapat diambil melalui metode bawaan .getMessage() pada saat exception ditangkap di blok catch.

B. class RuteTidakDitemukanException.

```
1 public class RuteTidakDitemukanException extends Exception {
2     public RuteTidakDitemukanException(String message) {
3         super(message);
4     }
5 }
```

Penjelasan: Kelas RuteTidakDitemukanException dirancang sebagai Custom Exception untuk mendeteksi kondisi ketika pengguna memasukkan kode kereta yang tidak terdaftar di dalam memori sistem.

1. Pewarisan Kelas Exception (Baris 1): Kata kunci extends Exception mengategorikan kelas ini sebagai Checked Exception. Berbeda dengan Unchecked, compiler Java mewajibkan setiap metode yang berpotensi melempar exception ini untuk mendeklarasikannya menggunakan throws, atau membungkusnya dalam blok try-catch di sisi pemanggil. Hal ini tercermin pada deklarasi metode prosesPemesanan() di kelas PusatKontrolReservasi.
2. Penerusan Informasi Galat (Baris 2-4): Konstruktor menerima parameter String message dan meneruskannya ke kelas induk (Exception) melalui instruksi super(message). Mekanisme ini memastikan rantai pelacakan kesalahan terjaga, sehingga sistem dapat menyampaikan pesan informatif tentang kode kereta mana yang gagal ditemukan.

C. Class TiketHabisException.

```
1 public class TiketHabisException extends Exception {
2     private String namaKereta;
3     private int sisaKursi;
4
5     public TiketHabisException(String message, String namaKereta, int sisaKursi) {
6         super(message);
7         this.namaKereta = namaKereta;
8         this.sisaKursi = sisaKursi;
9     }
10
11     public String getNamaKereta() {
12         return namaKereta;
13     }
14
15     public int getSisaKursi() {
16         return sisaKursi;
17     }
18 }
```

Penjelasan: Kelas TiketHabisException dirancang sebagai Custom Checked Exception yang tidak hanya membawa pesan error, tetapi juga menyimpan data kontekstual berupa nama kereta dan sisa kursi saat transaksi ditolak.

1. **Karakteristik Checked Exception dan Variabel Enkapsulasi (Baris 1-3):** Kelas mewarisi Exception sehingga bersifat Checked Exception. Dua atribut privat, namaKereta dan sisaKursi, ditambahkan untuk menyimpan informasi status kereta pada saat error terjadi. Enkapsulasi atribut ini menjaga integritas data agar hanya dapat diakses melalui metode getter yang disediakan.
2. **Inisialisasi Berlapis pada Konstruktor (Baris 5-9):** Konstruktor menerima tiga parameter: message, namaKereta, dan sisaKursi. Instruksi super(message) pada baris pertama konstruktor mengeset pesan error ke kelas induk. Selanjutnya, kata kunci this digunakan untuk memetakan parameter ke variabel instansiasi objek, sehingga data kontekstual melekat pada objek exception.
3. **Metode Getter (Baris 11-17):** Kelas menyediakan dua metode getter publik, yaitu getNamaKereta() dan getSisaKursi(). Penyediaan getter ini memungkinkan blok catch di kelas JavaExpress untuk mengekstrak informasi detail dan menampilkan pesan yang informatif kepada pengguna, seperti sisa kursi yang benar-benar tersedia.

D. Class KeretaApi

```
1 public class KeretaApi {
2     private String kodeKereta;
3     private String namaKereta;
4     private String rute;
5     private int kapasitas;
6     private int sisaKursi;
7
8     public KeretaApi(String kodeKereta, String namaKereta, String rute, int kapasitas) {
9         this.kodeKereta = kodeKereta;
10        this.namaKereta = namaKereta;
11        this.rute = rute;
12        this.kapasitas = kapasitas;
13        this.sisaKursi = kapasitas;
14    }
15
16    public String getKodeKereta() {
17        return kodeKereta;
18    }
19
20    public String getNamaKereta() {
21        return namaKereta;
22    }
23
24    public String getRute() {
25        return rute;
26    }
27
28    public int getKapasitas() {
29        return kapasitas;
30    }
31
32    public int getSisaKursi() {
33        return sisaKursi;
34    }
35
36    public void kurangiKursi(int jumlah) {
37        this.sisaKursi -= jumlah;
38    }
39
40    @Override
41    public String toString() {
42        return String.format("%-5s | %-15s | %-10s | %d/%d kursi tersedia",
43                               kodeKereta, namaKereta, rute, sisaKursi, kapasitas);
44    }
45 }
```

Penjelasan: Kelas KeretaApi bertindak sebagai kelas model data yang mengimplementasikan konsep Enkapsulasi, salah satu pilar fundamental Pemrograman Berorientasi Objek.

1. **Deklarasi Atribut Ber-modifier Private (Baris 2-5):** Seluruh atribut (kodeKereta, namaKereta, rute, kapasitas, sisaKursi) dideklarasikan private. Hal ini

menyembunyikan detail data internal dari akses langsung kelas eksternal sehingga integritas data terjaga dari modifikasi ilegal.

2. **Konstruktor Berparameter (Baris 7-13):** Konstruktor menerima empat parameter untuk inisialisasi awal objek. Perhatikan bahwa sisaKursi diinisialisasi sama dengan kapasitas (baris 12), sehingga saat kereta baru dibuat, seluruh kursi otomatis tersedia. Kata kunci `this` digunakan untuk membedakan variabel instansiasi dari parameter dengan nama yang sama.
3. **Metode Getter (Baris 15-19):** Karena atribut bersifat `private`, kelas menyediakan lima metode getter publik yang memberikan akses baca (read-only) kepada kelas luar. Pola ini memastikan data hanya dapat dimodifikasi melalui metode yang telah dirancang.
4. **Metode Prosedural kurangiKursi() (Baris 21-23):** Metode ini memanipulasi `sisaKursi` menggunakan operasi pengurangan majemuk (`--`). Metode ini yang dipanggil setelah validasi berhasil di kelas `PusatKontrolReservasi`, sehingga pengurangan kursi hanya terjadi bila seluruh syarat terpenuhi.
5. **Override toString() (Baris 25-28):** Metode `toString()` di-override untuk menghasilkan representasi String terformat menggunakan `String.format()`. Format ini digunakan oleh metode `lihatJadwal()` di kelas utama untuk menampilkan tabel jadwal yang rapi dan mudah dibaca di konsol.

E. Class PusatKontrolReservasi

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class PusatKontrolReservasi {
5     private List<KeretaApi> daftarKereta;
6
7     public PusatKontrolReservasi() {
8         daftarKereta = new ArrayList<>();
9     }
10    public void tambahKereta(KeretaApi kereta) {
11        daftarKereta.add(kereta);
12    }
13    public List<KeretaApi> getDaftarKereta() {
14        return daftarKereta;
15    }
16    public void prosesPemesanan(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)
17        throws RuteTidakDitemukanException, TiketHabisException {
18
19        if (nik.length() != 16) {
20            throw new DataPenumpangTidakValidException(
21                "NIK harus tepat 16 karakter! NIK Anda: " + nik.length() + " karakter.");
22        }
23        for (char c : nik.toCharArray()) {
24            if (!Character.isDigit(c)) {
25                throw new DataPenumpangTidakValidException(
26                    "NIK hanya boleh mengandung angka!");
27            }
28        }
29
30        KeretaApi keretaDipilih = null;
31        for (KeretaApi k : daftarKereta) {
32            if (k.getKodeKereta().equalsIgnoreCase(kodeKereta)) {
33                keretaDipilih = k;
34                break;
35            }
36        }
37
38        if (keretaDipilih == null) {
39            throw new RuteTidakDitemukanException(
40                "Kereta dengan kode '" + kodeKereta + "' tidak ditemukan!");
41        }
42
43        if (jumlahTiket > keretaDipilih.getSisaKursi()) {
44            throw new TiketHabisException(
45                "Kursi tidak mencukupi! Tiket diminta: " + jumlahTiket,
46                keretaDipilih.getNamaKereta(),
47                keretaDipilih.getSisaKursi());
48        }
49
50        keretaDipilih.kurangiKursi(jumlahTiket);
51        System.out.println("  PESANAN TIKET BERHASIL!");
52        System.out.println("  Nama Penumpang : " + namaPenumpang);
53        System.out.println("  NIK              : " + nik);
54        System.out.println("  Kereta          : " + keretaDipilih.getNamaKereta());
55        System.out.println("  Rute            : " + keretaDipilih.getRute());
56        System.out.println("  Jumlah Tiket    : " + jumlahTiket);
57        System.out.println("  Sisa Kursi      : " + keretaDipilih.getSisaKursi());
58    }
59 }
```

Penjelasan: Kelas PusatKontrolReservasi bertindak sebagai komponen Controller yang mengatur manipulasi data objek kereta di memori dan menegakkan aturan bisnis melalui pelemparan exception terstruktur.

1. **Inisialisasi Koleksi Data (Baris 7-8):** Atribut `daftarKereta` dideklarasikan menggunakan tipe antarmuka `List` dan diinstansiasi sebagai `ArrayList`. Pemilihan `ArrayList` didasarkan pada sifatnya yang dinamis dalam menampung objek di memori. Data awal kereta ditambahkan dari kelas `JavaExpress` melalui metode `tambahKereta()`.
2. **Validasi NIK dengan Unchecked Exception (Baris 22-30):** Metode `prosesPemesanan()` melakukan dua tahap validasi NIK: pertama mengecek panjang karakter menggunakan `.length() != 16`, lalu mengecek setiap karakter menggunakan `Character.isDigit()`. Jika salah satu gagal, `DataPenumpangTidakValidException` dilempar menggunakan kata kunci `throw`. Karena `Unchecked Exception`, metode tidak memerlukan deklarasi `throws` untuk exception ini.
3. **Pencarian Kereta dan Checked Exception Rute (Baris 32-41):** Pencarian dilakukan menggunakan perulangan `for-each` dengan `.equalsIgnoreCase()` agar tidak sensitif terhadap huruf kapital. Jika `keretaDipilih` masih `null` setelah perulangan selesai, maka `RuteTidakDitemukanException` dilempar. Exception ini wajib didaftarkan pada method signature menggunakan `throws` karena bersifat `Checked Exception`.
4. **Validasi Ketersediaan Kursi dan Checked Exception Tiket (Baris 43-48):** Tahap validasi akhir membandingkan jumlah `Tiket` dengan `getSisaKursi()`. Jika permintaan melebihi sisa kursi, `TiketHabisException` dilempar dengan menyertakan `namaKereta` dan `sisaKursi` sebagai data kontekstual. Pemanggil (`catch block`) dapat memanfaatkan getter-nya untuk memberikan informasi yang berguna kepada pengguna.
5. **Eksekusi Pemesanan (Baris 50-51):** Jika seluruh validasi berhasil dilewati, metode `kurangiKursi()` dipanggil untuk mengurangi sisa kursi, diikuti pencetakan struk pemesanan ke konsol. Desain ini memastikan perubahan data hanya terjadi setelah semua syarat bisnis terpenuhi.

F. Main

```

3 public class Main {
4     public static void main(String[] args) {
5
6         try {
7             catch (NumberFormatException e) {
8                 System.out.println("!! Error, harus masukkan angka.");
9             }
10
11         }
12
13         try {
14             tutupSystem();
15         } finally {
16             System.out.println("sistem ditutup");
17         }
18     }
19
20     static void tampilKeremu() {
21         System.out.println("=====");
22         System.out.println("1. Lihat Jadwal Kereta");
23         System.out.println("2. Pesan Tiket");
24         System.out.println("3. Keluar");
25         System.out.print("Pilih menu: ");
26     }
27
28     static void lihatJadwal() {
29         System.out.println("JADWAL KERETA TERSEDIA");
30         System.out.printf("%-5s | %-15s | %-10s | %-10s\n","kode", "Nama Kereta", "Rute", "kursi");
31         for (KeretaApi k : sistem.getJadwalKereta()) {
32             System.out.println(k);
33         }
34     }
35
36     static void pesanTiket() {
37         System.out.println("===== PEMESANAN TIKET =====");
38
39         try {
40             System.out.print("Kode Kereta : ");
41         }
42     }
43 }

```

```

01 //
02 //
03 System.out.print("Kode kereta : ");
04 String kodeKereta = scanner.nextLine().trim();
05 System.out.print("NIK harus 16 digit : ");
06 String nik = scanner.nextLine().trim();
07 System.out.print("Nama penumpang : ");
08 String nama = scanner.nextLine().trim();
09 System.out.print("Jumlah tiket : ");
10 int jumlahTiket;
11 try {
12     jumlahTiket = Integer.parseInt(scanner.nextLine().trim());
13 } catch (NumberFormatException e) {
14     System.out.println("[!] jumlah tiket harus berupa angka.");
15     return;
16 }
17
18 sistem.proses Memesan (kodeKereta, nik, nama, jumlahTiket);
19
20 catch (DuratPenumpangTidakValidException e) {
21     System.out.println("[ERROR] Data tidak valid : " + e.getMessage());
22 }
23 catch (RuteTidakDitemukanException e) {
24     System.out.println("[ERROR] Rute tidak ditemukan : " + e.getMessage());
25 }
26 catch (TiketTidakTersediaException e) {
27     System.out.println(" " + e.getMessage());
28 }
29 System.out.println(" Kereta : " + e.getNamaKereta());
30 System.out.println(" Sisa kursi : " + e.getSisaKursi());
31 }
32
33 static void tutupSistem() {
34     System.out.println("Data dihapus");
35 }

```

Penjelasan: Kelas JavaExpress bertindak sebagai titik masuk program yang mengintegrasikan seluruh kelas model, controller, dan exception ke dalam satu kesatuan alur program berbasis CLI.

1. **Inisialisasi Data Awal (Baris 7-9):** Dua objek `KeretaApi` (Argo Bromo dan Parahyangan) ditambahkan ke sistem melalui `tambahKereta()` saat program pertama dijalankan. Pendekatan ini memastikan data sudah ada di memori sebelum pengguna berinteraksi dengan menu.
2. **Loop Utama dengan Boolean Flag (Baris 11-25):** Perulangan `while(running)` dengan flag boolean `running` menjadi kendali utama program. Loop berjalan terus selama `running` bernilai `true` dan hanya berhenti ketika pengguna memilih menu 3 (Keluar). Struktur ini lebih aman dibanding `while(true)` karena memberikan kondisi terminasi yang jelas.
3. **Penanganan `NumberFormatException` dan Pembersihan Buffer (Baris 20-22):** Ketika pengguna mengetik bukan angka pada menu, `Integer.parseInt()` melempar `NumberFormatException`. Karena `Scanner.nextLine()` sudah digunakan (bukan `nextInt()`), buffer otomatis bersih setelah exception sehingga program tidak terjebak dalam infinite loop.
4. **Multi-catch Block di `pesanTiket()` (Baris 30-39):** Metode `pesanTiket()` menerapkan penanganan error berlapis. Urutan `catch` dimulai dari yang paling spesifik: `DataPenumpangTidakValidException` (format NIK), `RuteTidakDitemukanException` (kode kereta tidak ada), hingga `TiketHabisException` (kursi habis). Blok `TiketHabisException` memanfaatkan getter kustom untuk menampilkan nama kereta dan sisa kursi secara real-time.
5. **Blok Finally yang Dijamin Tereksekusi (Baris 27-28):** Blok `try-finally` pada penutupan program memastikan `tutupSistem()` dan pesan konfirmasi selalu dieksekusi, bahkan jika terjadi error tak terduga saat proses penutupan. Ini merupakan praktik yang baik untuk menutup sumber daya seperti koneksi database atau file stream.

V. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada sistem reservasi tiket kereta api "java express", dapat disimpulkan beberapa hal sebagai berikut.

- Penerapan arsitektur Error Handling secara terstruktur menggunakan blok try-catch mampu meningkatkan ketahanan aplikasi terhadap error waktu berjalan (fault tolerance). Program tidak berhenti mendadak (crash) ketika pengguna memasukkan data yang salah, melainkan menangkap error dan menampilkan pesan yang informatif.
- Integrasi antara Custom Checked Exception (RuteTidakDitemukanException dan TiketHabisException) dengan Unchecked Exception (DataPenumpangTidakValidException) efektif dalam memisahkan logika bisnis utama dari logika penanganan kesalahan, sehingga kode lebih terstruktur dan mudah dipelihara.
- Desain TiketHabisException yang membawa data kontekstual (namaKereta dan sisaKursi) melalui atribut privat dan getter menunjukkan bagaimana Custom Exception dapat dimanfaatkan tidak hanya sebagai sinyal error, tetapi juga sebagai pembawa informasi yang berguna bagi pengguna akhir.
- Penggunaan scanner.nextLine() pada seluruh input, dikombinasikan dengan Integer.parseInt(), terbukti mencegah infinite loop akibat input mismatch yang umum terjadi saat menggunakan nextInt() bersama Scanner.
- Blok finally pada penutupan sistem menjamin eksekusi kode pembersihan sumber daya, mencerminkan praktik pemrograman yang bertanggung jawab terhadap manajemen memori dan sumber daya sistem.

VI. Daftar Pustaka

Schildt, H. (2018). Java: The Complete Reference (11th ed.). McGraw-Hill Education.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: Error Handling. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.